

03 · Interrupts & Timer

Mikrocomputertechnik 3 · Vom Hardware-Tick zur Ereignissteuerung

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 80

Willkommen zu Block 3!

- Willkommen zur dritten Vorlesung in Mikrocomputertechnik 3
- Rückblick Block 1 und 2: Speicherphysik und Caches
- Bisher: Daten möglichst schnell zur CPU bringen
- Heute: Zeit und asynchrone Ereignisse
- Praxis-Fundament: System-Timer als Basis jedes Betriebssystems

Notizen

Herzlich willkommen zurück! In den letzten beiden Einheiten haben wir das Speicher-Dilemma gelöst. Sie wissen nun, wie Code und Variablen physikalisch abgelegt und durch Caches beschleunigt werden. Aber ein System-on-Chip muss mehr können, als nur schnell Daten zu verarbeiten. Er muss auf die reale Welt reagieren. Heute lernen wir, wie ein SoC komplexe Ereignisse verwaltet und wie wir der CPU ein Gefühl für die Zeit beibringen.

Folie

2 / 80

Einordnung im Semester



Notizen

Block 3 knüpft an Speicher und Caches an und öffnet das Thema Echtzeit: Timer, Interrupts und Ereignissteuerung.

Anknüpfung: Cache und Lokalität

Kurz-Recap aus Block 2 (Labor folgt mit dem Übungsblatt):

- Spaltenweiser Zugriff kann den Cache durch schlechte Lokalität belasten
- Zeilenweiser Zugriff maximiert räumliche Lokalität und Hit Rate
- Software und Hardware sind untrennbar verbunden

```
// Cache-freundliches Muster aus Block 2
for (int row = 0; row < 1000; row++) {
    for (int col = 0; col < 1000; col++) {
        bild[row][col] = 0;
    }
}
```

Notizen

In Block 2 haben wir gesehen, dass die Struktur Ihres C-Codes die Hit Rate massiv beeinflusst. Das Cache-Labor (z. B. mit Ripes) vertieft das messbar — sobald das Übungsblatt vorliegt. Behalten Sie die Denkweise: Wir programmieren mit der Hardware, nicht gegen sie.

Von Daten zu Ereignissen

- Eine CPU bei 3 GHz ist extrem schnell
- Die Außenwelt (Taster, Sensoren) ist vergleichsweise langsam
- Warten in einer Schleife verschwendet Energie
- Ziel: nützliche Arbeit, während auf Peripherie gewartet wird
- Ausweg: asynchrone, ereignisgesteuerte Systeme

Notizen

Was nützt die schnellste Pipeline, wenn das Programm in einer Schleife auf einen Tastendruck wartet? Die CPU soll rechnen und nur reagieren, wenn wirklich etwas passiert.

CPU-Zeit sinnvoll nutzen

Phase	Inhalt (schematisch)
Main	nützliche Arbeit
ISR	kurzes Event-Handling
Main	weiterrechnen

Interrupt unterbricht nur kurz — kein Dauer-Polling.

Notizen

Statt Busy-Waiting: Die CPU arbeitet im Main-Loop und wechselt nur bei Ereignissen kurz in die Interrupt Service Routine (ISR).

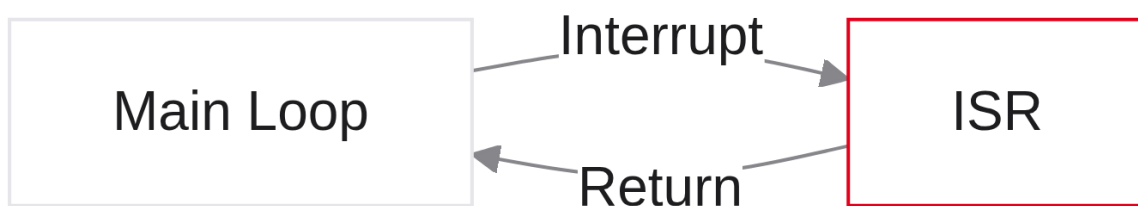
Refresher MCT1: Polling vs. Interrupts

- Polling (Busy Waiting): ständiges Abfragen — hohe Last, kein sinnvolles Multitasking
- Interrupt: „Türklingel-Prinzip“
- Hardware signalisiert Ereignis → CPU unterbricht die laufende Arbeit
- Danach Rückkehr in den unterbrochenen Kontext

Notizen

Polling ist „Sind wir schon da?“ — stromhungrig und starr. Interrupts lassen die CPU rechnen, bis die Hardware klingelt.

Polling vs. Interrupt — Schema



Notizen

Schematisch: Das Main-Programm läuft weiter, bis ein Interrupt die ISR anstößt; danach Rückkehr.

Folie

8/80

Refresher MCT1: Das CSR-System

- Control and Status Registers: spezielle Maschinenregister
- `mstatus`: globaler Interrupt-Schalter (u. a. MIE)
- `mcause`: Grund der Unterbrechung (Interrupt/Exception-Code)
- `mepc`: Rücksprungadresse des unterbrochenen Programms
- `mret`: Rückkehr anhand von `mepc`

```
# Typische Aktion in einer ISR (Kontext lesen)
csrr t0, mcause      # Warum wurde ich geweckt?
csrr t1, mepc        # Wo muss ich später wieder hin?
```

Notizen

Ohne CSRs könnte die Hardware den Interrupt-Kontext nicht verwalten. Bei einem Trap schreibt die Hardware den Grund nach `mcause` und merkt sich in `mepc` den Fortsetzungspunkt.

Folie

9/80

Der rudimentäre MCT1-Trap-Handler

- In MCT1 oft ein einziger Einsprung über `mtvec`
- Alle Quellen landen an derselben Adresse
- Software muss per `mcause` verzweigen (`if/else`)
- Für wenige Quellen akzeptabel — für echte SoCs zu starr und langsam

```
// Logischer Ablauf des einfachen Handlers
void trap_handler(void) {
    int cause = read_csr(mcause);
    if (cause == BUTTON) handle_button();
    else if (cause == TIMER) handle_timer();
}
```

Notizen

Ein gemeinsamer Trap-Handler reicht für Übungen mit wenigen Quellen. Bei dutzenden Peripherien wird die Software-Suche zum Flaschenhals.

Agenda für den heutigen Tag

1. Grenzen des einfachen Traps
2. Vektorisierte Interrupts (Vectored Mode)
3. RISC-V CLINT — lokale Timer und Software-Interrupts
4. RISC-V PLIC — Dirigent für externe Peripherie
5. Nested Interrupts
6. Brückenschlag zum Labor (System-Ticks, z. B. in Renode)

Notizen

Heute rüsten wir auf: direkter Sprung zur passenden ISR, Hardware-Timer und ein Controller für viele externe Quellen.

Lernziele Block 3

- Erklären, warum lineare `if/else`-Ketten in ISRs für SoCs ungeeignet sind
- Vektor-Tabelle und Vectored Mode in RISC-V verstehen
- CLINT (lokal) und PLIC (global) unterscheiden
- Mathematik hinter dem 64-Bit-System-Timer (`mtime`) beherrschen
- Gefahren und Lösungen bei Nested Interrupts kennen

Notizen

Am Ende verstehen Sie, wie moderne Chips Ereignisse verarbeiten — und Sie beherrschen den Hardware-Timer als Herzstück für Scheduler und Zeitmessung.

Die Leitfrage des Tages

- Moderne MCUs: dutzende Peripheriegeräte können Interrupts auslösen
- Theoretisch sogar im selben Taktzyklus
- Wer darf zuerst? Was passiert mit den anderen?
- Leitfrage: Wie managt die CPU viele Alarmer ohne Suchaufwand und ohne den Wichtigen zu verpassen?
- Antwort: spezialisierte Interrupt-Controller-Hardware

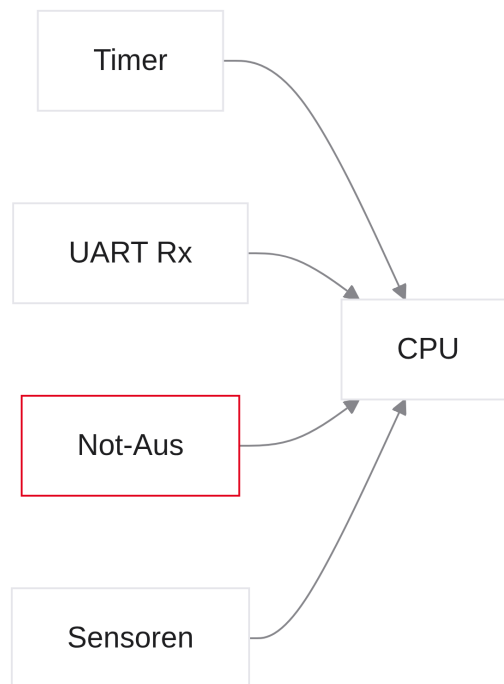
Notizen

Viele Ereignisse gleichzeitig — wie Feueralarm, Telefon und Klingel auf einmal. Die CPU braucht ein intelligentes Verteilzentrum vor dem Kern.

Folie

13 / 80

Viele Quellen — ein Problem



Notizen

Schematisch: Viele Quellen drängen zur CPU. Ohne Priorisierung und schnelle Identifikation entsteht Chaos — besonders bei kritischen Signalen wie Not-Aus.

Folie

14 / 80

Das Skalierungsproblem

- MCT1: wenige Quellen (z. B. Taster, ein Timer)
- Echte SoCs: dutzende Interrupt-Quellen
- RISC-V-Kern: nur wenige feste Interrupt-Leitungen (u. a. Timer, Software, External)
- Folge: viele Geräte teilen sich typischerweise den External-Interrupt-Pfad

Notizen

Der Kern bleibt schlank: nicht hundert Pins, sondern wenige standardisierte Interrupt-Eingänge. Externe

Der Software-Flaschenhals

- Alle teilen sich „External“ → `mcause` sagt oft nur die Kategorie
- Software muss Quellen abfragen: „Warst du das?“
- Lange `if/else`-Ketten kosten viele Takte

```
// Problematisch für Echtzeit
void isr(void) {
    if (check(SPI)) handle_spi();
    else if (check(I2C)) handle_i2c();
    /* ... viele weitere Abfragen ... */
}
```

Notizen

Bei gebündelten Quellen reicht „Externes Ereignis“ nicht aus. Sequenzielle Software-Suche skaliert schlecht.

Latenz-Explosion

- Latenz: Zeit vom Signal bis zur nützlichen ISR
- Best Case: Quelle steht oben in der Kette
- Worst Case: viele falsche Abfragen zuvor
- Komplexität $\mathcal{O}(n)$ — inakzeptabel für harte Echtzeit (z. B. ABS)

Notizen

Schwankende Reaktionszeiten sind in Embedded-Echtzeitsystemen problematisch. Wir brauchen möglichst deterministische Latenzen.

Das Prioritäten-Problem

- Gleichzeitige Events: wer gewinnt?
- Reine Software-Kette: Priorität oft „wer steht oben im Code“
- Schwer zur Laufzeit flexibel umzupriorisieren
- Wunsch: Hardware entscheidet den Gewinner automatisch

Notizen

Statische Code-Reihenfolge ist starr. Kritische Alarmer müssen unabhängig von der Schreibreihenfolge im C-Code gewinnen können.

Die RISC-V-Architektur-Lösung

- Arbeitsteilung: CPU rechnet, Controller managen Alarmer
- Externe Module sammeln, priorisieren und melden den Gewinner
- Direkter Sprung zur passenden Software-Routine
- Trennung: lokale vs. globale Ereignisse

Notizen

Statt den Flaschenhals in C zu programmieren, übernimmt dedizierte Hardware das Sortieren und Weiterreichen.

Interrupt-Controller vor der CPU



Notizen

Schematisch: Der Controller filtert und priorisiert; die CPU sieht den ausgewählten Alarm.

Lokale vs. globale Interrupts

Lokale Interrupts (core-nah):

- Beispiele: System-Timer, Software-Interrupts
- Eigene Leitungen in den RISC-V-Kern
- Sehr geringe Latenz

Globale Interrupts (Peripherie):

- UART, SPI, Taster, ...
- Über den External-Interrupt-Pfad gebündelt

Notizen

Lokale Ereignisse gehören eng zur CPU (z. B. OS-Tick). Globale kommen von der Plattform-Peripherie.

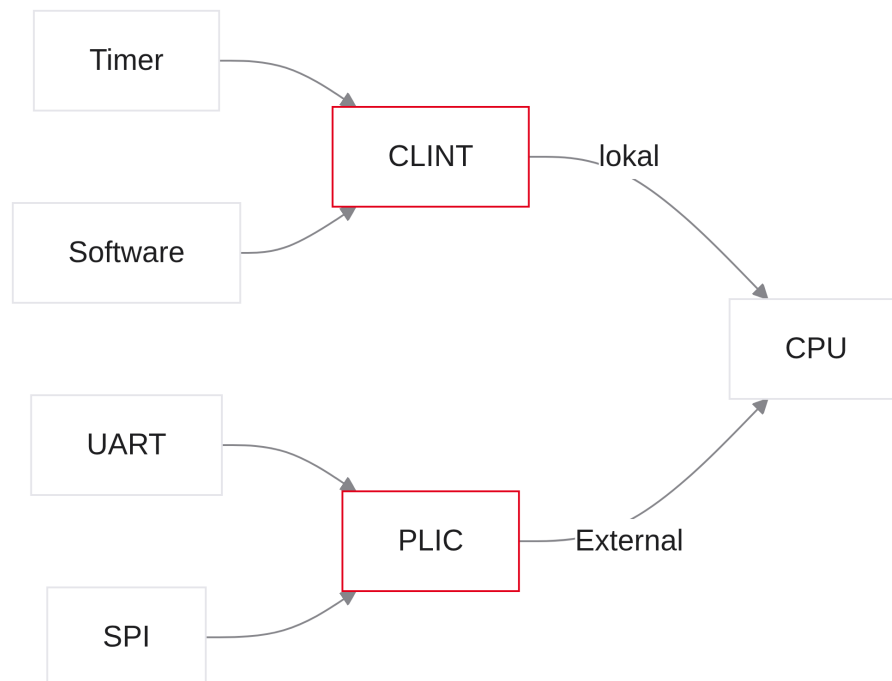
Die zwei Giganten: CLINT und PLIC

- CLINT (Core Local Interruptor): lokale Timer- und Software-Interrupts
- PLIC (Platform-Level Interrupt Controller): Dirigent für externe Geräte
- PLIC → External Interrupt an die CPU

Notizen

CLINT und PLIC strukturieren das SoC-Design: Zeit lokal, Peripherie global.

CLINT und PLIC — Schema



Notizen

Architektur für den Rest der Vorlesung: CLINT für Zeit, PLIC für die Plattform. Zuerst lösen wir das Suchproblem per Vectored Mode.

Folie

23 / 80

Fragen? — Motivation

- Skalierungsproblem klar?
- Warum sind lineare `if/else`-Ketten in ISRs riskant für Echtzeit?
- Lokal vs. global?

Notizen

Wenn die Grenzen des MCT1-Handlers klar sind, folgt die Lösung: Vektor-Tabellen.

Das Register `mtvec` im Detail

- `mtvec`: Basisadresse der Trap-/Interrupt-Behandlung
- In MCT1 oft vereinfacht als reine Handler-Adresse gesetzt
- Unterste 2 Bits sind MODE — keine Adressbits
- Obere Bits: BASE-Adresse

Bits 31:2	Bits 1:0
BASE	MODE

Notizen

`mtvec` ist mehr als eine Adresse: Die unteren Bits konfigurieren das Eintrittsverhalten.

Das MODE-Feld in `mtvec`

- MODE = 0 (Direct): alle Traps zur Adresse BASE (MCT1-Verhalten)
- MODE = 1 (Vectored): Interrupts springen versetzt; Exceptions weiter zu BASE
- MODE 2 und 3: im Standard reserviert

Notizen

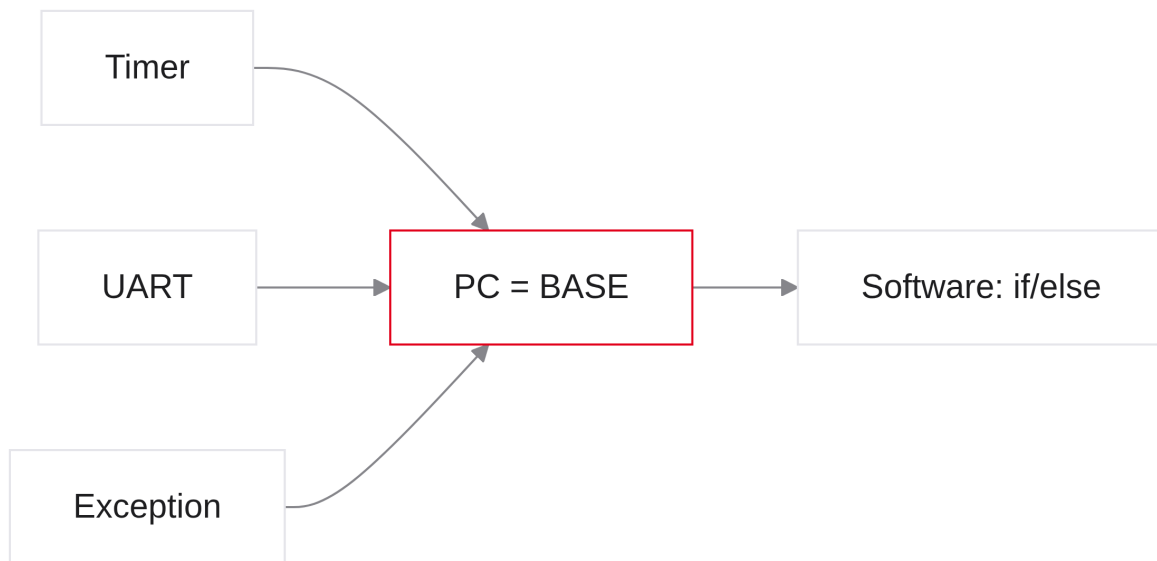
Vectored Mode lässt die Hardware den Einsprung anhand der Ursache berechnen — ohne Software-Suche.

Direct Mode (MODE = 0)

- Hardware setzt PC = BASE
- Alle Quellen landen an derselben Stelle
- Software liest `mcause` und verzweigt

Notizen

Wie eine Hotline mit Sprachmenü: erst sortieren, dann bearbeiten.

Direct Mode — Schema**Notizen**

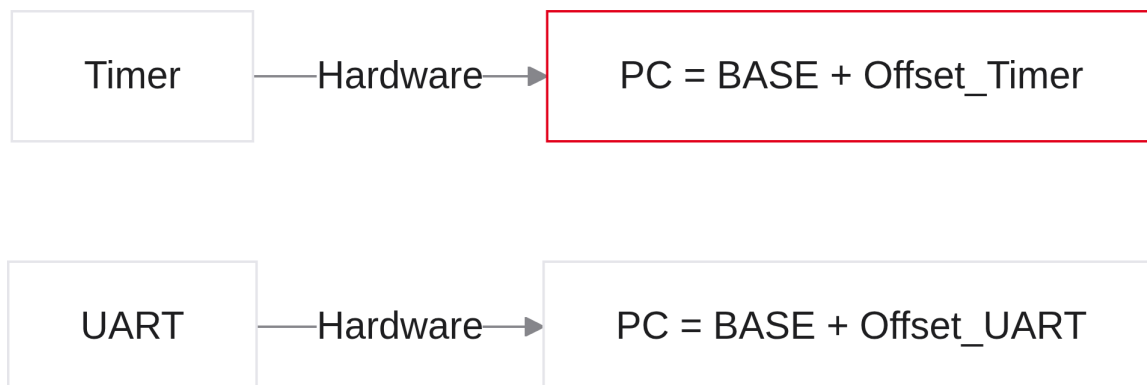
Alle Pfade treffen sich in einem Trichter — einfach, aber nicht skalierbar.

Vectored Mode (MODE = 1)

- Jeder Interrupt-Typ erhält einen eigenen Einsprung
- Hardware nutzt den Exception-Code aus `mcause`
- Kein `if/else` zur Identifikation der Quelle
- Sprung in konstanter Zeit zur passenden Routine

Notizen

Direkte Durchwahl statt Hotline: Timer → Timer-ISR, UART → UART-ISR.

Vectored Mode — Schema**Notizen**

Die Hardware berechnet den Ziel-PC aus BASE und Interrupt-Code.

Wie Vectored Mode rechnet

Bei MODE = 1 und Interrupt:

$$PC = BASE + 4 \times mcause.ExceptionCode$$

- Faktor 4: ein RV32I-Befehl belegt typischerweise 4 Bytes
- Beispiel: Timer-Interrupt (Code 7) → $BASE + 28$

Notizen

Silizium multipliziert den Code mit der Befehlslänge und addiert zur Basis — deterministisch und schnell.

Die Vektor-Tabelle

- An BASE liegt eine Tabelle aus Sprungbefehlen (`j / jal`)
- Eintrag k bei Offset $4k$
- Beispiel: bei Offset 28 der Sprung zur Timer-ISR

```
.align 4
vector_table_base:
    j isr_cause_0      # Offset 0
    j isr_cause_1      # Offset 4
    # ...
    j isr_timer         # Offset 28 (Code 7 * 4)
```

Notizen

Die Tabelle ist die Absprungrampe: Hardware landet auf dem passenden Jump, der in Ihren C-Handler führt.

Memory Layout einer Vektor-Tabelle

- Oft früh im Flash (nahe Boot-Code), Section z. B. `.vectors`
- RISC-V Vectored Mode: an `BASE + 4k` liegt ein **Befehl** (`j/jal`), keine Adresse
- Gegensatz zu Cortex-M: dort speichert die Tabelle Funktionszeiger (Daten)
- In C: Sprungtabelle typisch in Assembler oder als Inline-Asm/Linker-Skript

```
.section .vectors, "ax"
.align 2
vector_table_base:
    j isr_cause_0
    j isr_cause_1
    /* ... */
    j isr_timer          /* Offset 28 = 7 * 4 */
```

Notizen

Wichtiger Unterschied zur ARM-Welt: Die Hardware führt die Instruktion an der Vektoradresse aus. Ein Array von Funktionszeigern wäre nur Daten — die CPU würde Adressen als Maschinenbefehl interpretieren. Deshalb: Jump-Tabelle, nicht Zeiger-Tabelle.

Alignment-Vorgaben

- Unterste 2 Bits von `mtvec` = MODE
- BASE muss an den unteren Bits 0 sein (sonst Mode-Kollision)
- Tabelle mindestens 4-Byte-aligned; je nach System strengere Grenzen
- In Assembler: `.align`

Notizen

Eine ungerade BASE würde MODE-Bits verfälschen. Alignment ist Pflicht, kein Detail.

Der Performance-Gewinn

- Ohne Vektoren: Suchlatenz $\mathcal{O}(n)$
- Mit Vektoren: Identifikation in Hardware, $\mathcal{O}(1)$
- Reaktionszeit unabhängig von der Position in einer Software-Liste
- Grundlage für harte Echtzeit

Notizen

Egal ob Quelle 1 oder 50: der Einsprung bleibt kurz und vorhersagbar.

Fragen? — Vectored Mode

- Direct vs. Vectored klar?
- Warum $\times 4$?
- Vektor-Tabelle verstanden?

Notizen

Als Nächstes: die Hardware, die den Timer-Pfad speist — der CLINT.

Was ist der CLINT?

- CLINT = Core Local Interruptor
- Standardisiertes Modul nahe der Pipeline
- Kein generischer ISA-Befehl, sondern Systembaustein
- Aufgabe: lokale Zeit (Timer) und Software-Interrupts
- Typische Codes: Machine Timer (7), Machine Software (3)

Notizen

Neben ALU und Register braucht das SoC Module wie den CLINT, die Zeit zählen und den Kern gezielt wecken.

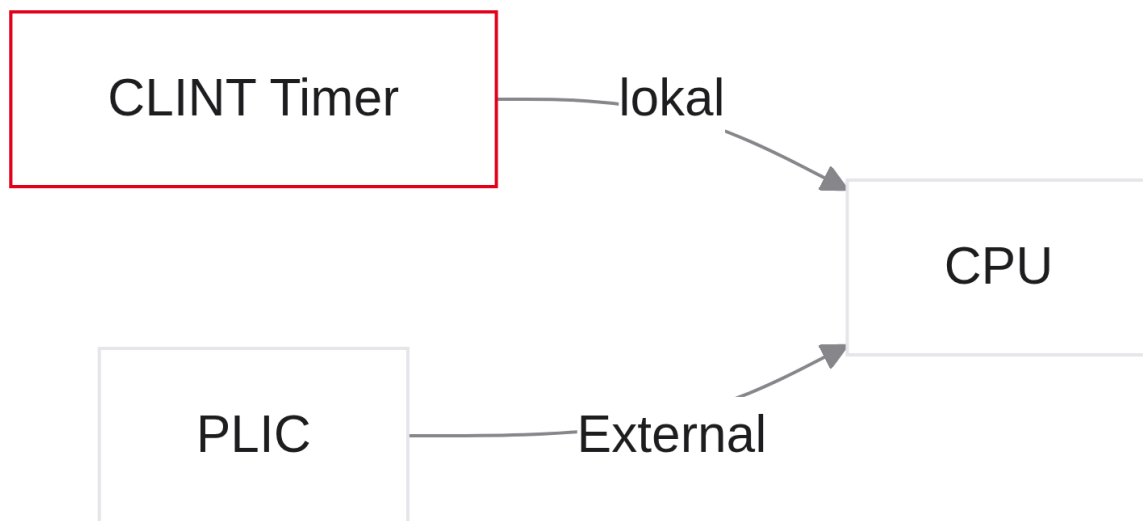
Warum „Local“?

- Timer-Interrupts sind oft kritisch für OS (Scheduler, Context Switch)
- Sollen nicht hinter UART/SPI auf einem gemeinsamen Bus warten
- CLINT hat eine direkte, priorisierte Anbindung an den Kern

Notizen

Der System-Tick ist der Herzschlag des OS — deshalb VIP-Pfad statt Peripherie-Stau.

CLINT — lokale Überholspur



Notizen

Schematisch: CLINT lokal, PLIC über den External-Pfad.

Folie

39 / 80

Das Herz der Zeit: `mtime`

- Register `mtime` (Machine Time) im CLINT
- Zählt mit dem Takt eines Hardware-Oszillators
- Läuft, sobald der Chip mit Spannung versorgt ist
- Repräsentiert System-Uptime (seit Start)

Notizen

`mtime` ist die unaufhaltsame Systemuhr — Software stoppt sie typischerweise nicht.

Das 64-Bit-Problem auf RV32

- RV32: native Wortbreite 32 Bit
- 32-Bit-Zähler bei 1 MHz: Overflow nach ≈ 71 Minuten
- Deshalb: `mtime` im CLINT 64 Bit — auch auf RV32
- Overflow erst nach astronomisch langen Zeiten (bei üblichen Takten)

Notizen

Eingebettete Systeme laufen oft monatelang. Eine 32-Bit-Uhr würde Zeitmessung und Scheduler zerstören.

64-Bit lesen auf RV32

- CLINT liefert `mtime` als `mtime_lo / mtime_hi`
- Race: Overflow zwischen den beiden Reads möglich
- Sicheres Muster: `hi` vor und nach `lo` prüfen

```
uint32_t hi, lo;
do {
    hi = CLINT_MTIME_HI;
    lo = CLINT_MTIME_LO;
} while (hi != CLINT_MTIME_HI);
```

Notizen

Klassisches Embedded-Muster gegen Tear: solange `hi` stabil ist, war der 64-Bit-Wert konsistent.

Der Auslöser: `mtimecmp`

- `mtimecmp` (Machine Time Compare), ebenfalls 64 Bit
- Software schreibt einen Zukunftswert (Eieruhr)
- Beispiel: jetzt = 100, Intervall 50 $\rightarrow$ `mtimecmp` = 150

Notizen

Wir lesen `mtime`, addieren das Intervall und programmieren den Alarm — dann weiter im Main-Loop.

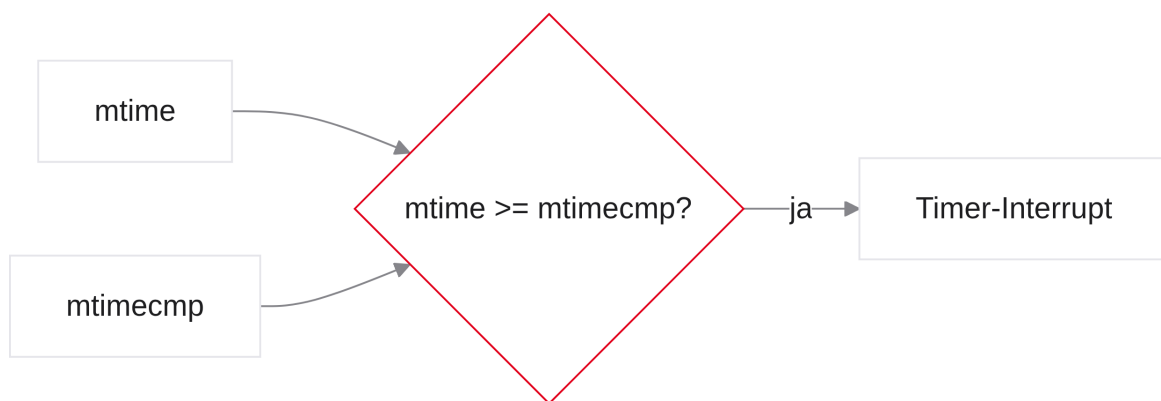
Hardware-Logik im CLINT

- 64-Bit-Komparator: in jedem Takt `mtime >= mtimecmp?`
- Bei wahr: lokaler Timer-Interrupt an die CPU
- CPU springt (im Vectored Mode) zur Timer-ISR

Notizen

Der Komparator arbeitet dauernd in Hardware — kein Polling in Software nötig.

CLINT-Komparator — Schema



Notizen

Schematisch: Zähler gegen Zielwert — bei Erreichen Interrupt.

Quittieren des Timer-Interrupts

- Vergleich ist $\geq$: Interrupt bleibt aktiv, solange `mtime >= mtimecmp`
- Ohne Update: sofort erneuter Eintritt nach `mret`
- Pflicht in der ISR: neuen Zukunftswert nach `mtimecmp` schreiben

```
void timer_isr(void) {
    uint64_t now = read_mtime();
    write_mtimecmp(now + INTERVAL); /* neu scharf schalten */
    /* eigentliche Arbeit ... */
}
```

Notizen

Häufiger Anfängerfehler: LED toggeln und zurückkehren, ohne mtimecmp zu aktualisieren — dann feuert der Alarm sofort wieder. Besser `mtimecmp += INTERVAL` statt `now + INTERVAL`, um Drift durch ISR-Latenz zu vermeiden.

Folie

46 / 80

Konzept: Der System-Tick

- Wofür nutzen wir den Timer in der Praxis?
- System-Tick (SysTick): Herzschlag eines OS (z. B. FreeRTOS, Linux)
- Periodischer Alarm (typisch jede 1 ms)
- In der ISR: globale Zeitbasis hochzählen (z. B. `system_uptime_ms`)
- Scheduler: bei Tick prüfen, ob Task A pausiert und Task B läuft (preemptives Multitasking)

Notizen

Der CLINT-Timer ist mehr als ein Wecker: Er ist der Herzschlag des Systems. Multitasking auf einem Kern beruht darauf, dass der Hardware-Timer die CPU regelmäßig unterbricht und der Scheduler den Kontext wechselt.

Folie

47 / 80

Warum oft 1 Millisekunde?

- SysTick in Embedded Systems oft 1 ms (1000 Hz)
- Kompromiss: Auflösung vs. Overhead
- Zu grob (z. B. 1 s): Software kann nicht fein reagieren
- Zu fein (z. B. 1 µs): ISR-/Kontextwechsel-Overhead dominiert
- 1 ms: bewährter Sweetspot für Wahrnehmung und CPU-Last

Notizen

Jeder Interrupt kostet Pipeline- und Registeraufwand. Bei 1 µs-Ticks bleibt kaum Zeit für Nutzarbeit. Bei 1 ms hat eine CPU mit z. B. 100 MHz noch etwa 10^5 Takte für echte Berechnungen — und für Menschen wirkt 1 ms noch „sofort“.

Mathematik: Takte berechnen

- `mtime` zählt Oszillator-Takte, keine Sekunden
- Umrechnung in Software nötig
- Beispiel: Quarz 10 MHz $\Rightarrow 10^7$ Takte/s
- Pro ms: $10^7/1000 = 10^4$ Takte
- Nächster Alarm: `mtimecmp = mtime + 10000`

```
#define TIMER_FREQ    10000000 /* 10 MHz */
#define TICKS_PER_MS (TIMER_FREQ / 1000)

void timer_isr(void) {
    uint64_t now = read_mtime();
    write_mtimecmp(now + TICKS_PER_MS);
}
```

Notizen

Im Datenblatt steht die Timer-Frequenz. Daraus berechnen Sie Ticks pro Millisekunde und programmieren `mtimecmp` entsprechend — das ist typische Labor-Mathematik.

Software-Interrupts im CLINT

- Register `msip` (Machine Software Interrupt Pending)
- Ein Bit pro Kern: Schreiben einer 1 $\rightarrow$ Soft-Interrupt (Exception Code 3)
- Sinn: Arbeit aus einer hochpriorigen ISR in einen niedriger priorisierten Pfad auslagern (Deferred Interrupt Handling)

Notizen

Die CPU kann sich selbst „anklingeln“. Oft: in der schnellen ISR nur das Nötigste, Soft-Interrupt setzen, Rest später abarbeiten.

Inter-Processor Interrupts (IPI)

Vorgriff auf Block 9 (Multicore):

- Ein CLINT, mehrere `msip`-Bits (eines pro Kern)
- Kern A schreibt `msip` von Kern B → asynchroner Interrupt auf B
- Standardweg für Inter-Processor Communication / Aufwecken

Syntax error in text
mermaid version 11.12.0

Notizen

Schlafende Kerne können so gezielt geweckt werden — z. B. nach Netzwerkempfang auf Kern 0 die Entschlüsselung auf Kern 1 anstoßen.

MMIO beim CLINT

- `mtime / mtimecmp` sind keine CSRs (`csrw`), sondern Memory-Mapped Register
- Zugriff über normale C-Pointer auf physische Adressen
- Basisadresse abhängig vom SoC (bei SiFive oft ab `0x02000000`)

Notizen

CSRs steuern den Kern von innen; der CLINT liegt außerhalb auf dem Bus — wie Peripherie über MMIO.

C-Beispiel: SysTick initialisieren

```
#define CLINT_MTIMECMP 0x02004000
#define CLINT_MTIME    0x0200BFF8

/* RV32: mtime atomar hi/lo lesen (kein 64-Bit-Load!) */
static uint64_t read_mtime(void) {
    volatile uint32_t *mt = (volatile uint32_t *)CLINT_MTIME;
    uint32_t hi, lo;
    do {
        hi = mt[1];
        lo = mt[0];
    } while (hi != mt[1]);
    return ((uint64_t)hi << 32) | lo;
}

void init_systick(void) {
    uint64_t now = read_mtime();
    *(volatile uint64_t *)CLINT_MTIMECMP = now + TICKS_PER_MS;
    set_csr(mie, (1u << 7)); /* MTIE */
    set_csr(mstatus, MSTATUS_MIE); /* MIE global */
}
```

- MTIE in mie: Timer-Interrupt-Quelle freigeben
- MIE in mstatus: Interrupts global erlauben
- Beide Schalter nötig — sonst bleibt der Alarm stumm

Notizen

Rezept: Zeit lesen, Intervall addieren, Compare setzen — dann MTIE und MIE. Adressen sind beispielhaft (SiFive-üblich); immer Datenblatt prüfen.

Zusammenfassung CLINT

- Lokale, zeitkritische Ereignisse
- 64-Bit-Zähler `mtime`, Compare `mtimecmp`
- Soft-Interrupts (`msip`): Deferred Handling und IPI
- Konfiguration über MMIO

Notizen

Damit haben wir die Systemzeit. Als Nächstes: externe Peripherie über den PLIC.

Fragen? — CLINT und SysTick

- Warum 64-Bit `mtime`?
- Race Condition beim 64-Bit-Lesen auf RV32?
- Warum sofort erneuter Timer-Interrupt ohne neues `mtimecmp`?

Notizen

Wenn Timer und Quittieren klar sind, öffnen wir die Tür zur Peripherie: PLIC.

Auftritt PLIC

- PLIC = Platform-Level Interrupt Controller
- Dirigent für dutzende externe Geräte
- Sammelt Signale, priorisiert, meldet ein External Interrupt an den Kern

Notizen

Die Sortierlogik wandert von langen `if/else`-Ketten in dedizierte Hardware.

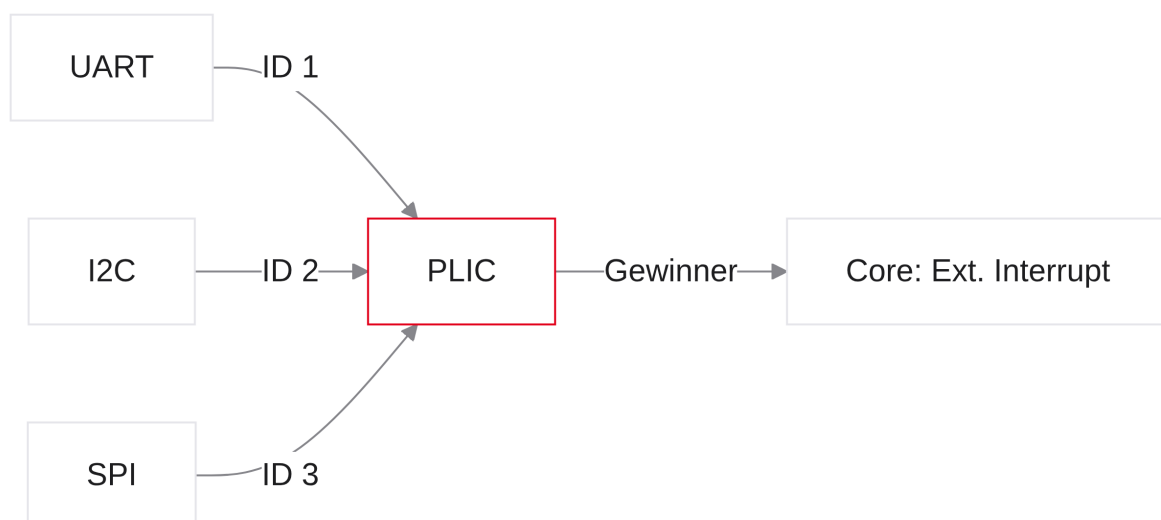
PLIC-Architektur

- Gateways: Anbindung der Sensoren/Peripherie
- Arbitration: Schiedsrichter über Prioritäten
- Targets: typischerweise CPU-Kerne
- M:N-Router (viele Quellen → wenige Kerne)

Notizen

Egal ob 10 oder 100 Quellen: der Kern sieht weiterhin den External-Pfad — das hält die Pipeline schlank.

PLIC — Schema



Notizen

Schematisch: mehrere Quellen, ein External Interrupt zum Kern — Identifikation folgt per Claim.

Zuweisung von Prioritäten und Enables

- Jedes Gerät hat eine ID
- Pro Gerät: Priority-Register (MMIO); typisch 0 = aus, größer = höher
- Zusätzlich: Enable-Bits pro Target — ohne Enable bleibt die Quelle stumm
- Prioritäten und Enables zur Laufzeit per Software änderbar

Notizen

Priorität allein reicht nicht: die Quelle muss für den Zielkern auch enabled sein. Beides steuern Sie über MMIO-Register im PLIC.

Priority Thresholds

- Pro Target (Kern): Threshold-Register
- Weiterleitung nur wenn Priorität **strikt größer** als Threshold
- Beispiel: Threshold = 5 → Prio 3 wird blockiert, Prio 7 weckt die CPU

Notizen

Nützlich, wenn die CPU kritische Arbeit schützt: niedrige Alarme prallen am PLIC ab, Notfälle kommen durch.

Das Pending-Array

- Blockierte/wartende Quellen gehen nicht verloren
- Pending-Bit pro Quelle — bleibt gesetzt bis Claim
- Arbitrer wählt parallel die höchste Priorität über dem Threshold
- Gewinner → External Interrupt an die CPU

Notizen

Klingeln wird gespeichert. Sinkt der Threshold später, kann der PLIC ausstehende Pendants nachreichen. Die Auswahl selbst läuft in Hardware-Gattern — ohne $\mathcal{O}(n)$ -Software-Suche.

Zwei-Stufen-Handshake

- External Interrupt: `mcause` sagt oft nur „PLIC / External“ (Code 11)
- CPU kennt die Quellen-ID noch nicht
- Pflicht: Claim (ID lesen) und später Complete (fertig melden)

Notizen

Ohne Handshake weiß die Software nicht, wer gewonnen hat — und der PLIC kann den Kanal nicht korrekt freigeben.

Schritt 1: Claim

```
void external_interrupt_handler(void) {  
    uint32_t id = *(volatile uint32_t *)PLIC_CLAIM_REG;  
    my_isr_functions[id]();  
}
```

- MMIO-Read auf Claim-Register → Gewinner-ID
- Danach direkter Sprung (z. B. Funktionszeiger-Array)

Notizen

Ein Read ersetzt die lange Abfragekette: ID lesen, passende ISR aufrufen.

Atomares Lesen und Löschen

- Claim-Read löscht atomar das zugehörige Pending-Bit
- In Multicore: nur ein Kern erhält die ID; andere lesen 0
- Verhindert doppelte Bearbeitung derselben Quelle

Notizen

Atomarität verhindert Race Conditions, wenn mehrere Kerne gleichzeitig claimen.

Claim und Complete

```
void external_interrupt_handler(void) {
    uint32_t id = *(volatile uint32_t *)PLIC_CLAIM_REG;
    my_isr_functions[id]();                /* Treiberarbeit */
    *(volatile uint32_t *)PLIC_CLAIM_REG = id; /* Complete */
}
```

- Während Claimed: dieselbe Quelle blockiert; andere dürfen pending werden
- ID zurückschreiben → Kanal wieder freigeben
- Danach `mret` in den unterbrochenen Kontext

Notizen

Ohne Complete bleibt die Quelle oft dauerhaft blockiert. Claim und Complete teilen sich typischerweise dieselbe Registeradresse (Read vs. Write).

PLIC in Software steuern (MMIO)

- Prioritäten, Enables, Threshold, Claim/Complete — alles MMIO
- Keine speziellen CSR-Befehle: Pointer in C
- Trigger am Gateway: Level (Leitung High) oder Edge (Flanke) — oft im Gerät konfigurierbar

Notizen

Wie beim CLINT: Treiber schreiben Integer-Werte auf feste Adressen. Level-Signale bleiben aktiv, bis die Ursache im Gerät gelöscht ist; Edge reagiert auf den Spannungswechsel.

Gegenüberstellung: CLINT vs. PLIC

	CLINT (lokal)	PLIC (global)
Zuständigkeit	Timer, Soft/IPI	UART, SPI, GPIO, ...
Latenz	minimal (core-nah)	etwas höher (Systembus)
Prioritäten	fest/kritisch	frei konfigurierbar
Handshake	<code>mtimecmp</code> neu setzen	Claim und Complete

Notizen

SysTick → CLINT; viele Sensoren → PLIC. Das ist die Merksatz-Trennung für Klausur und Praxis.

Folie

67 / 80

Fragen? — PLIC

- Pending vs. Threshold klar?
- Warum Claim/Complete zwingend?
- Warum atomarer Claim-Read?

Notizen

Als Nächstes: Darf eine ISR von einer wichtigeren ISR unterbrochen werden?

Folie

68 / 80

Standardverhalten: „taube“ CPU

- Bei Trap setzt die Hardware `MIE` in `mstatus` automatisch auf 0
- Während der ISR: weitere Interrupts blockiert bis `mret`
- Schützt vor Chaos — Nachteil bei langen ISRs

Notizen

Standard: Ohren zu, ISR ohne Störung abarbeiten. Sicher, aber in Echtzeitsystemen oft zu restriktiv.

Folie

69 / 80

Problem blockierender ISRs

- Langsame ISR (z. B. I2C, 5 ms) bei SysTick alle 1 ms
- Mit `MIE = 0` verpassen wir mehrere Ticks
- Zeitbasis und Scheduler leiden
- Regel: ISRs so kurz wie möglich halten

Notizen

Lange ISRs blockieren den Herzschlag. Besser: Flag setzen, Arbeit in die Main-Schleife verschieben (Deferred Processing).

Lösung: Nested Interrupts

- Nesting: eine laufende ISR darf von einer höher prioren unterbrochen werden
- Beispielablauf: Main → I2C (niedrig) → Timer (hoch) → weiter I2C → Main
- PLIC kann höherpriorie Quellen durchreichen — die CPU muss Nesting aber explizit erlauben (MIE)
- Mächtig für Echtzeit, softwareseitig aufwendig

Notizen

Die CPU stapelt Ereignisse nach Priorität. Das schützt die Zeitbasis, erfordert aber korrekte Register-Rettung — sonst geht der Rücksprung verloren.

Clobbering von mepc

- Hardware hat nur **ein** mepc
- Main → I2C: mepc = Rücksprung Main
- Nesting + Timer: Hardware überschreibt mepc mit I2C-PC
- Main-Rücksprung verloren → Absturz

Notizen

Bei verschachtelten Traps überschreibt die Hardware das einzige mepc. Ohne vorherige Sicherung in Software ist der Weg zurück in die Main-Schleife verloren.

Nesting korrekt: zuerst Stack, dann MIE

```

addi sp, sp, -8      /* Platz fuer mepc + mstatus */
csrr t0, mepc
sw    t0, 0(sp)      /* mepc retten */
csrr t1, mstatus
sw    t1, 4(sp)      /* mstatus retten */
/* erst jetzt: MIE = 1 und unterbrechbare Arbeit */
/* ... innere Arbeit ... */
lw    t1, 4(sp)      /* mstatus zurueck */
csrw mstatus, t1
lw    t0, 0(sp)      /* mepc zurueck */
csrw mepc, t0
addi sp, sp, 8
mret

```

- Reihenfolge: Stack-Platz → CSRs sichern → dann MIE setzen
- Nach dem inneren Interrupt: Werte zurückschreiben, dann `mret`
- MIE=1 ohne Stack-Rettung ist unsicher

Notizen

Klassische OS-Technik: Hardware-CSRs in RAM sichern, bevor ein zweiter Trap sie überschreiben darf. Erst danach die globalen Interrupts wieder freigeben.

Folie

73/80

Gefahren von Nested Interrupts

- Stack Overflow bei tiefer Verschachtelung
- Deadlocks zwischen ISRs (Locks / geteilte Ressourcen)
- Praxis: Nesting nur wenn nötig; ISRs kurz halten; Arbeit deferred in `main`

Notizen

Industrie-Empfehlung: Nested Interrupts sparsam. Flag setzen, Main erledigt die Arbeit — oft robuster und debugbarer.

Fragen? — Nesting

- Warum überschreibt Hardware `mepc` beim Nesting?
- Wie schützt man sich vor Clobbering?
- Warum Nesting möglichst vermeiden?

Notizen

Wenn Stack-Rettung und Clobbering klar sind, spannen wir den Bogen zum Labor.

Ausblick Labor: SysTick-Szenario

Geplantes Laborziel:

- CLINT so konfigurieren, dass er periodisch (z. B. 1 ms) feuert
- In der ISR `system_uptime_ms` hochzählen
- `main`-Schleife steuert Verhalten über diese Zeitbasis (z. B. LED-Takt)
- Hardwarenaher C-Code für RISC-V (Simulation in Renode)

Notizen

Im Labor steht der CLINT-Herzschlag im Vordergrund — nicht der volle PLIC-Treiber. Details folgen im Übungsblatt.

Renode und die Zeit

- Renode simuliert den CLINT-Quarz deterministisch
- Konfigurierte 1 ms $\Leftrightarrow$ korrekte Anzahl virtueller Takte
- Unabhängig von der Last des Host-PCs
- Verhalten wie auf dem Zielchip (bis auf Modellgenauigkeit)

Notizen

Deshalb Renode: virtuelle Millisekunden sind an Taktzyklen gebunden, nicht an Windows-Scheduling.

GCC: `__attribute__((interrupt))`

- ISR muss mit `mret` enden, nicht mit normalem `ret`
- Attribut: `__attribute__((interrupt))` — GCC erzeugt Prolog/Epilog
- Normale ABI: Callee darf `t`-Register zerstören; eine ISR darf keine Spuren hinterlassen
- Mit dem Attribut: berührte Register (inkl. temporärer) werden gesichert

```
__attribute__((interrupt))
void timer_isr(void) {
    /* ... */
}
```

Notizen

Erinnerung MCT1: Calling Conventions. Eine ISR ist kein normales Unterprogramm — der Compiler muss den Kontext schützen, damit die unterbrochene `main()` konsistent weiterläuft.

Background Task und Foreground ISR

- Foreground: kurze ISR inkrementiert `system_uptime_ms` (nur lesen/schreiben der Zeitbasis)
- Background: `main` vergleicht gegen einen Merker — ohne die ISR-Zählung zu nullen

```
volatile uint32_t system_uptime_ms;
uint32_t last_toggle = 0;

while (1) {
    uint32_t now = system_uptime_ms;
    if (now - last_toggle >= 1000) {
        toggle_led();
        last_toggle = now;
    }
}
```

Notizen

Kein blockierendes `delay()`: die CPU bleibt in der Main-Schleife nutzbar. Die ISR liefert nur die Uhr; `main` speichert den letzten Aktionszeitpunkt separat — so entsteht keine Race durch `system_uptime_ms = 0`.

Geteilte Variablen

- Compiler-Caching: `volatile` für von ISR geschriebene, in `main` gelesene Variablen
- Atomarität: 64-Bit auf RV32 kann „reißen“ — beim Lesen ggf. kurz Interrupts sperren

Notizen

Zwei Laborregeln: `volatile` für Shared State; bei Mehrwort-Werten kritische Abschnitte beachten.

Zusammenfassung Block 3

- CLINT: Systemzeit (`mtime`) und Soft/IPI — niedrige Latenz; Freigabe über `MTIE` und `MIE`
- PLIC: viele externe Quellen, Priorität, Enable, Claim/Complete
- Vectored Mode: Sprungtabelle mit Instruktionen, $\mathcal{O}(1)$ -Einsprung
- Nested Interrupts: mächtig, aber `mepc` vorher auf den Stack retten
- GCC: `__attribute__((interrupt))` für Kontext-Sicherung

Vielen Dank — wir sehen uns im Labor.

Notizen

Damit ist das Fundament für Echtzeit und Ereignissteuerung gelegt. Nächster Schritt: den CLINT in Renode konfigurieren und darauf aufbauen (z. B. PWM).